Boosting - 1

17 October 2025 20:18

Agenda → Recap [D. Trees → Shallow → Bagging
D. Trees → Deep Trees

Today → what is Boosting ? →

(Story Time)

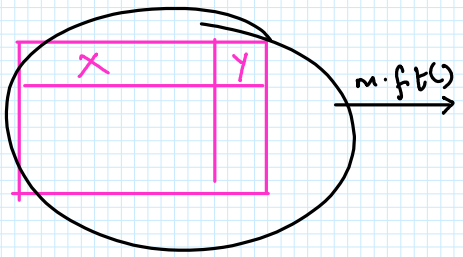
(Variance) D. Trees (village) → R.F (CTO)
D. Trees → Boosting (CEO)

Clarify → H.W
New algo → (Prev datasets) You should apply that new algo on all prev datasets

Process

Data → EDA → Scaling → Modeling
(same in all algos)
Modeling → (2 lines of code)

C1 — ( )
C2 — ( )
C3 — ( )
C4 — ( )



Last few sess

[{ (D. Trees) I — (D. Trees) (II) }]

Bagging (Applicn) → is the solution to variance problem

Boosting (2 classes) → Next class [Stronger]

(Bias → error)

Decision Trees

Deep D. Tree

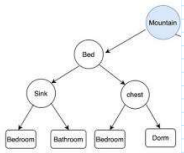


Low Bias High Variance

→ To overcome High Variance

Solution ⇒ (Bagging)

Shallow D-Tree



(High Bias) Low Variance

↓

(so we have to reduce Bias)

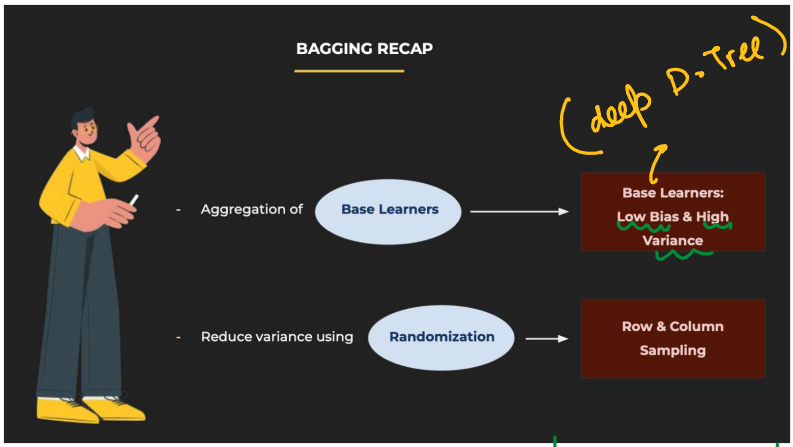
Boosting → To reduce Bias

RF → A **random forest** is a collection of **many decision trees** — a *forest*!
Each tree makes its own prediction, and the forest takes a **vote**.

* Each tree is independent

Why is it "random"?
Because when building each tree, we add randomness in two ways:

1. **Random subset of data:** each tree is trained on a random sample of the training data.
2. **Random subset of features:** each tree only looks at a few random features (like weather, temperature, or wind) at each split.



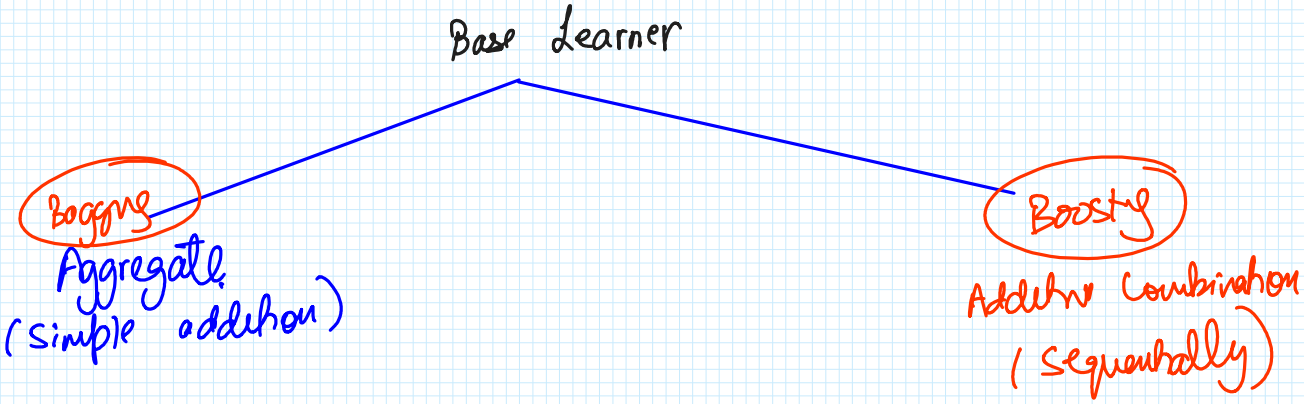
(deep D-Tree)

Base Learner	Bias	Variance	Soln
① Bagging → Deep D-Tree (w.l)	↓	↑	Bagging
② Boosting → Shallow Tree (weak Learner)	↑	↓	Boosting

(2) Boosting → Shallow "C" (weak Learner)

Q what is Boosting ? →

Can we combine "multiple weak learner (shallow D-Tree) sequentially" to overall a "strong learner" ?



Boosting is an ensemble method: instead of training one big model, you train many weak learners (often shallow decision trees) sequentially, and combine them into a stronger model.

Key Idea (Boosting) →

① start with simplest model (M_0)

② Each new model tries to correct the mistakes from all prev models

$err_2 = y - F_2$ *

$F_{100} = \underbrace{M_0 + \gamma_1 h_1 + \gamma_2 h_2}_{F_2} + \cdots \gamma_{100} h_{100}$

$M_3 = [X, err]$

h_2

#	Height	Gender	Weight(y)
1	1.6	M	82
2	1.5	F	55
3	1.4	F	61
4	1.4	M	65

$\frac{82+55+61+65}{4} = \phi$

1.2 F

$\phi = M_0$

Boosting Intuition - How to combine base learners ?

Let's understand it using a **regression** example :

#	Height	Gender	Weight(y)
1	1.6	M	82
2	1.5	F	55
3	1.4	F	61
4	1.4	M	65

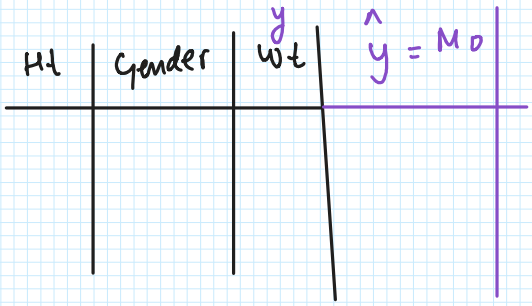
Given : Features

1. Height
2. Gender

Predict : Weight

$\{x^{(i)}, y^{(i)}\}_{i=1}^{n}$

On this **training data** , we build our model in step.



Let's recap using the regression example i.e predict weight using height & gender.

Height	Gender	Weight(y)
1.6	M	82
1.5	F	55
1.4	F	61
1.4	M	65

STEP 0: Mean Model

M0

$$h_0(x) = \bar{y}$$

$$= \frac{82 + 55 + 61 + 65}{4}$$

$$h_0(x) = 65.75$$

$$error^i = y^i - \hat{y}^i = y^i - 65.75$$

Height	Gender	Weight(y)	err_0
1.6	M	82	16.25
1.5	F	55	-10.75
1.4	F	61	-4.75
1.4	M	65	-0.75

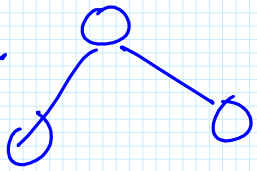
Which of the following model are underfit models ?

4 options
Active Duration(Most preferred: 30 seconds)

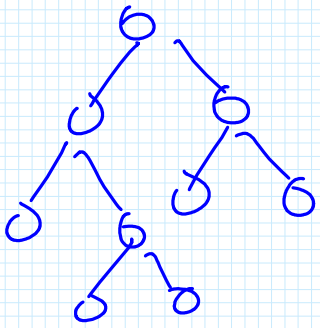
Appears for | 30 Secs

A	Decision Stump (Decision Tree with depth = 1)
B	Decision Tree with depth = 20
C	Decision Tree with depth = 10
D	None of the above.

Stump ≈



D. Tree



What model will have high bias low variance ? (Hint: think of simplest model)

3 options
Active Duration(Most preferred: 30 seconds)

Appears for | 30 Secs

A	Mean model (which predict mean value everytime)
B	Random model (model which predicts random value everytime)
C	A DT fit perfectly on training data

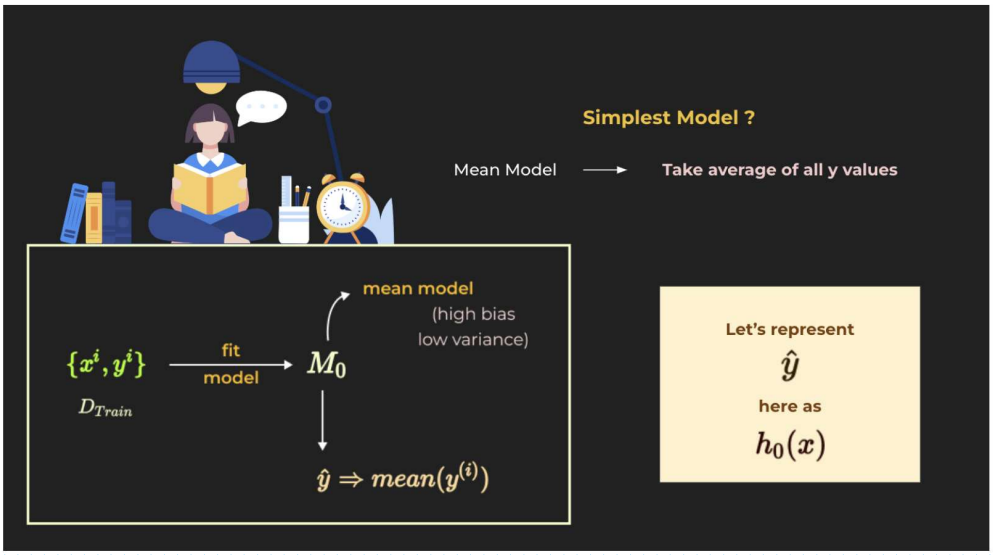
→ M_0 → starting step of boosting

Trainy data

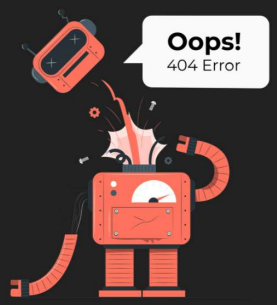
Boosting →

X_1	X_2	X_3	Y

Internally M_0 =



Each prediction will have an **error** associated with it



For every point,

$$err^{(i)} = y^{(i)} - \hat{y}^{(i)}$$

Actual

Predicted = $h_0(x)$

So $err^{(i)} = y^{(i)} - h_0(x)$

As it is a mean model → **Large Training Data**

STEP 1: M1

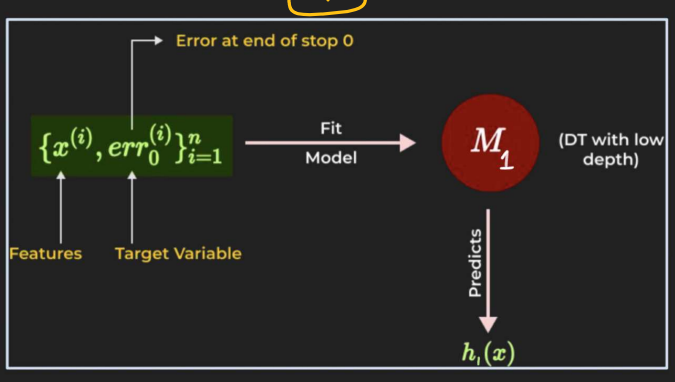
In this step, we fit a model (M1) on error of previous step (err_0^i)



What model shall we use as (M1) ?

- Can be any model as long as it is high bias low variance.
- We will consider **DECISION TREES**
- DT will be of low depth

The process will look like :



Now, we need to add prediction of step 0 & step 1

Additive Combination

The final model $F_1(x)$ (model at the end of Stage 1) will be defined as :

The final model $F_1(x)$ (model at the end of Stage 1) will be defined as :

$$F_1(x) = h_0(x) + h_1(x)$$

$\bar{y}$ + $h_1(x)$

Prediction at stage 0 — Prediction at stage 1



Issue with **simple** addition ?

There is a possibility that

- M_1 model has large residual
- Not contributing much to final prediction

We need a mechanism to control the model contribution in final prediction.

M_1

How to do that ?

→ Multiply the model pred with weight values

$$F_1(x) = h_0(x) + \gamma_1 h_1(x)$$

Weight

Prediction at stage 0 — Prediction at stage 1

Lower Residual ? → Good Model → Give high influence in final prediction → **Large weights**

Higher Residual ? → Give Low influence in final prediction → **assign small weights**

stop @ M_1

$\hat{y} = h_0 + \gamma_1 h_1$

$= F_1$

[D-T with low depth] → Bias ↑ Low Variance ↓

$err_1 = y - F_1(x)$

Step M_2

$\{x^{(i)}, err_1\}$ → fit a model M_2 → $h_2(x)$

(Add h_2 to F_1)

(Add h_2 to F_1)

$$F_2 = F_1 + \gamma_2 h_2 = h_0 + \gamma_1 h_1(x) + \gamma_2 h_2(x)$$

$$err_2 = y - F_2(x)$$

$$\{x^{(i)}, err_{(2)}\} \xrightarrow{\text{fit a model}} h_3(x)$$

Add h_3 to final pred

$$F_3 = F_2 + \gamma_3 h_3 = h_0 + \gamma_1 h_1 + \gamma_2 h_2 + \gamma_3 h_3$$

[M_0 & M_1]

Stage 2 → M_2 → $\{x^i, err_1\}$

$err_1 = y - F_1(x)$ ⇒ $y - F_1(x)$

What error shall we use here at stage 2 ?

4 options

Active Duration(Most preferred: 30 seconds)

Appears for: 30 Secs

A	Residual left after subtracting final model prediction (F_1(x)) from actual value
B	Residual left after subtracting Stage 1 model (M_1) from actual value
C	Residual left after subtracting Stage 0 model (M_0) from actual value
D	None of the above

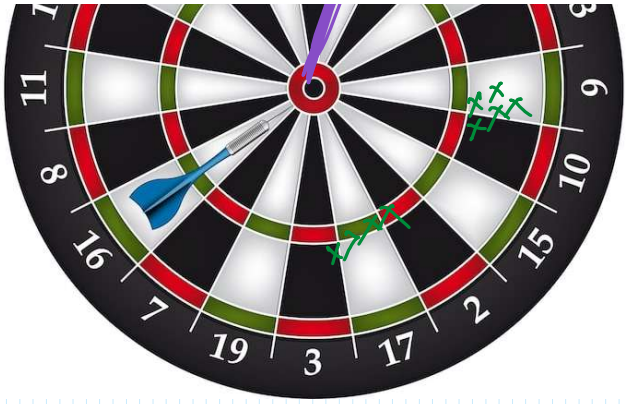
$F_7(x) = [h_0 + \gamma_1 h_1 + \cdots + \gamma_7 h_7]$

h_8 → $y - F_7 = error$



WK learner

high Bias Low variance



Vaari can I

high bias feature of the weak learner now becomes an advantage because now error becomes predictable and we can use them intelligently

(Summarise) → 22:14

Summarizing Process using an Example

Let's recap using the regression example i.e predict weight using height & gender.

Height	Gender	Weight(y)
1.6	M	82
1.5	F	55
1.4	F	61
1.4	M	65

STEP 0: Mean Model

$$h_0(x) = \bar{y}$$

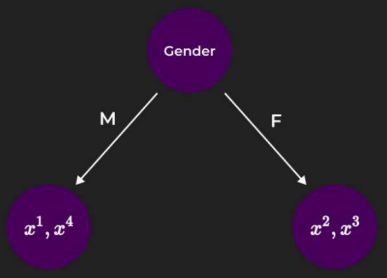
$$= \frac{82 + 55 + 61 + 65}{4}$$

$$h_0(x) = 65.75$$

Using the prediction of Stage 0 we calculate residual / error.

Height	Gender	Weight(y)	err_0
1.6	M	82	16.25
1.5	F	55	-10.75
1.4	F	61	-4.75
1.4	M	65	-0.75

Stage 1 : Using the residual (err_0^i) & features build a low depth DT



	Height	Gender	err_0	h1(x)
x1	1.6	M	16.25	7.75
x2	1.5	F	-10.75	-7.75
x3	1.4	F	-4.75	-7.75
x4	1.4	M	-0.75	7.75

$\hat{y}_{left} = \frac{16.25 - 0.75}{2}$ = 7.75

$\hat{y}_{right} = \frac{-10.75 - 4.75}{2}$ = - 7.75

$h_0 + \gamma_1 h_1$

Finally, combine the prediction of Stage 1 with Stage 0

f_1 better than f_0

Mo Model

γ_1 = 1 (Assume)

	Height	Gender	err_0	h1(x)	h0(x)	$f_1(x) = h_0(x) + \gamma_1 h_1$
x1	1.6	M	16.25	7.75	65.75	73.50
x2	1.5	F	-10.75	-7.75	65.75	58
x3	1.4	F	-4.75	-7.75	65.75	58
x4	1.4	M	-0.75	7.75	65.75	73.5

And using this final pred

Calculate the residual at end of Stage 1

Final pred at stage 1

	Height	Gender	Weight(y)	F1(x)	err_1
x1	1.6	M	82	73.1	8.5
x2	1.5	F	55	58	-3
x3	1.4	F	61	58	3
x4	1.4	M	73.5	73.5	-8.5

Model error after stage 1

$\frac{-\partial L}{\partial \hat{y}}$

- Using the err_1^i & x^i ,fit a new model 1 (i.e stage 2)
- This goes on till M steps.

How geometrically this works ?

How geometrically this works ?

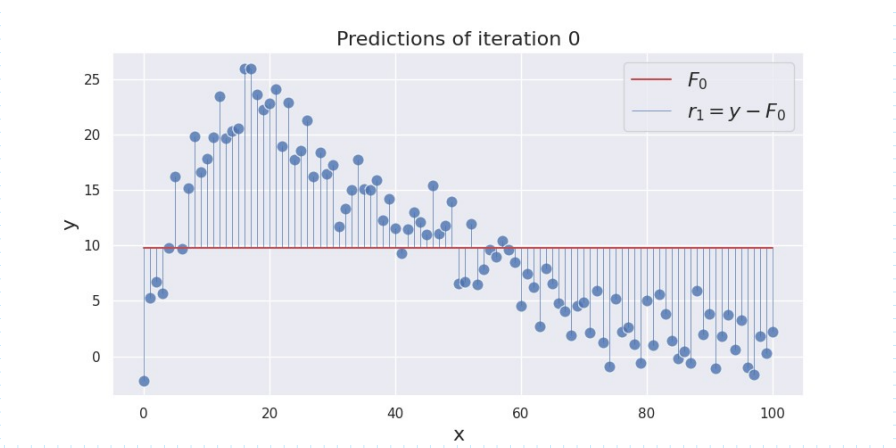
F_0 e_0

F_1 e_1

F_2 e_2

⋮ ⋮

F_{100} e_{100}



(#) Boosting → old

Fitting new decision trees on residuals of old

$(y - \hat{y})$

(Reg + clf) → can't directly work for clf

[(New loss functions develop)]

(MSE)

(MAE)

+ (Log Loss)

(MAE)

Want? → We can apply amazing boosting process to any loss function

Instead of fitting the next tree on residuals of (prev F) $(y - \hat{y})$

you fit it on "Pseudo Residuals"

↳ fn(loss function)

Regression $L(y, \hat{y}) = (y - \hat{y})^2$

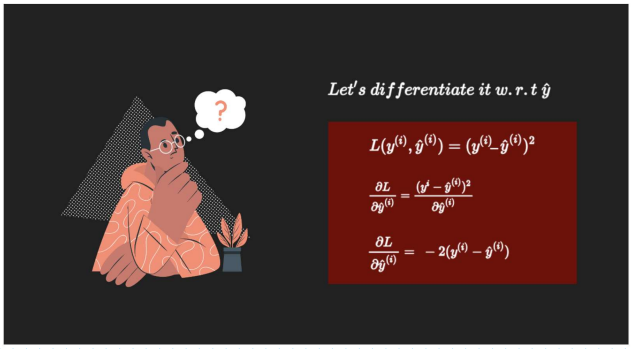
$$(-1)\frac{\partial L}{\partial \hat{y}} = 2(y - \hat{y})$$

$$-\frac{\partial L}{\partial \hat{y}} \propto (y - \hat{y}) = e$$

$(y - \hat{y})$ → Residual

(Pseudo Residual)

→ you can apply boosting to any loss fn



Let's multiply both sides with - 1

$$\frac{-\partial L}{\partial \hat{y}^{(i)}} = +2\,(y^{(i)} - \hat{y}^{(i)})$$

-ve gradient of loss w.r.t model output

Residual

If we were to ignore 2 on R.H.S

- Residual is proportional to -ve gradient of loss function w.r.t model prediction

We know,

$$\hat{y} = F_k(x^i)$$

Replacing $\hat{y}$ with $F_k(x^i)$ in last equation

$$\frac{-\partial L}{\partial F_k(x^i)} = +2\,(y^{(i)} - \hat{y}^{(i)})$$

-ve gradient of loss w.r.t model output

Residual

Note:

If it were classification problems, simply replace square loss with log loss.

These -ve gradients are called Pseudo Residual.

$$\left(x,\ \frac{-\partial L}{\partial F_R}\right)$$

Residual & Pseudo Residual

How do we use Pseudo Residual ?

($M_0 - - M_{j-1}$ are build)

Say we are building a model & currently at step j i.e Mj

We need $\{x^{(i)}, err^{(i)}\}$ to train m_j

$M_j \leftarrow \{x^{(i)}, err^{(i)}\}$

$h_j(x)$

$err^{(i)} = y^{(i)} - F_{j-1}(x^i)$

$err^{(i)} = residual$



Instead of calculating residual, simply calculate pseudo residual

L → already known

data known

F_{j-1} is also known

Currently, we are using

$$err^{(i)} = y^{(i)} - F_{j-1}(x^i)$$

$$err^{(i)} = residual$$

Instead use,

$$err^{(i)} \approx pseudo\ residual$$

$$= \frac{-\partial L}{\partial F_{j-1}}(x^i)$$

How does pseudo residual help us ?

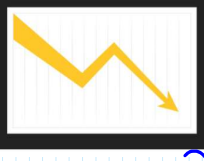
We are building model at each step → To minimize the residual

If we were to use pseudo residuals instead of residuals, → Indirectly minimizing the loss functions

Note: - Can minimize any custom loss function

- As long as it is differentiable





Cornell = (Kilian) *

(Using pseudo Residual is something like gradient Descent but in functional space)

POINTS TO REMEMBER

- Base Learners → High Bias & Low Variance (Stumps)
- Combine learners using **additive combining.**
- **Step 0 : M_0** → **Mean Model**
- Iteratively build models on the error of previous model (Pseudo Residual) (**Sequential process**)
- Use weights to adjust the influence of model in final prediction.

$F_{100} = h_0 + \gamma_1 h_1 + \ldots + \gamma_{100} h_{100}$

- **At train time :** Fit DTs & find weights (γ)
- **At test time :** Simply pass query point through base learner & multiply prediction with weight value to get final prediction
- **Why Boosting ?** - Can minimise any loss function [Tr error ↓ → use Boosting]
- **How does it minimize any loss function ?** - Using pseudo residuals

